

AI-BASED STUDY PLANNER

Rational Agent – Individual Project Report

Individual Technical Focus: Utility/Priority Rule, Deadline Urgency, Difficulty, Topic Weight and Rational Scheduling

Project Objective

Build a rational study-planning agent that accepts subjects, deadlines, difficulty levels, topic weights and available daily study hours, then generates a readable weekly timetable that prioritises urgent and high-weight topics.

1. ABSTRACT

Study planning is an important application of intelligent decision-making because students often have multiple subjects, different deadlines and limited study time. This project implements an AI-Based Study Planner as a rational agent. The agent accepts subject information, deadline, difficulty, topic weight and available daily study hours. It calculates a priority value for each subject using a utility rule and schedules subjects in descending priority order. The main focus is the rational decision rule used to select which subject should be studied first.

2. INTRODUCTION

A rational agent selects an action that is expected to achieve the best outcome according to a defined objective. In this study planner, the agent observes the available subjects and their characteristics, evaluates their urgency and importance, and creates a study schedule. The actual implementation uses the current date to calculate days remaining until each deadline. Difficulty and topic weight increase priority, while a closer deadline increases urgency.

3. PROBLEM STATEMENT

Develop an AI-based study planning system that takes subjects, deadlines, difficulty levels and available daily study hours as inputs and generates a weekly study timetable. The system must use a configurable objective such as maximising coverage of high-priority topics before deadlines. It must demonstrate the utility/priority rule used to choose what to schedule and explain why the resulting plan is rational given the inputs.

4. OBJECTIVES

- Accept multiple subjects and their deadlines.
- Consider difficulty and topic weight for every subject.
- Calculate days remaining until each deadline.
- Compute a rational priority score for scheduling.
- Sort subjects according to priority.
- Generate a readable weekly timetable based on available daily hours.
- Display the priority order so the agent's decisions can be understood.

5. RATIONAL AGENT DESIGN

The system can be viewed as a simple rational agent. Its percepts are subject name, deadline, difficulty, topic weight and available study hours. Its decision mechanism calculates a priority score and selects the highest-priority unscheduled subject. Its action is to assign the selected subject to a study slot. The objective is to give earlier attention to topics that are both important and difficult, especially when their deadlines are close.

6. UTILITY / PRIORITY RULE

The implemented priority rule is:

$$\text{Priority} = (\text{Difficulty} \times \text{Topic Weight}) / \text{Days Remaining}$$

Difficulty and topic weight are entered on a 1–5 scale. A smaller number of days remaining produces a larger priority value, representing greater urgency. Therefore, a subject with high difficulty, high weight and

a close deadline receives a higher priority than a less urgent subject.

7. RATIONALITY OF THE RULE

The rule is rational with respect to the stated objective because it combines three relevant factors: academic effort, importance and deadline urgency. Increasing difficulty raises the score, increasing topic weight raises the score, and decreasing days remaining raises the score. The agent therefore tends to allocate earlier study slots to subjects where the potential benefit of early attention is greater under the chosen objective.

8. WORKED PRIORITY EXAMPLE

Suppose a subject has difficulty 5, topic weight 5 and 2 days remaining.

$$\text{Priority} = (5 \times 5) / 2 = 12.5$$

Another subject with difficulty 3, weight 4 and 4 days remaining has:

$$\text{Priority} = (3 \times 4) / 4 = 3.0$$

The first subject is scheduled earlier because its calculated priority is higher under the defined rule.

9. ALGORITHM / PROCEDURE

1. Get today's date.
2. Read the number of subjects.
3. For each subject, read name, deadline, difficulty and topic weight.
4. Convert the deadline into a date value.
5. Calculate days remaining.
6. Prevent division by zero by treating deadlines due today or already passed as one day remaining.
7. Calculate the priority score.
8. Store the subject and its priority.
9. Sort subjects in descending priority order.
10. Read available study hours per day.
11. Assign one-hour study slots across up to seven days.
12. Display the weekly plan and final priority order.

10. IMPLEMENTATION

The implementation is written in Python and uses the built-in **datetime.date** class for date calculations. Subjects are stored as dictionaries inside a list. Python's sorting function orders the subjects using the calculated priority value. The weekly schedule uses the available daily-hour limit and assigns one hour to each selected subject.

11. CONFIGURABLE OBJECTIVE

The objective represented by the current implementation is to prioritise urgent and high-weight topics while considering difficulty. The main adjustable factors are the input values and available daily hours. The priority formula can also be changed in a future version if the objective changes, for example by assigning different importance to deadline urgency, difficulty or topic weight.

12. SAMPLE INPUT AND OUTPUT

Example input:

Subjects: 3

Mathematics — deadline 2026-09-19 — difficulty 5 — weight 5

Python — deadline 2026-09-22 — difficulty 4 — weight 4

English — deadline 2026-09-25 — difficulty 2 — weight 2

Available study hours per day: 2

Illustrative output structure:

===== WEEKLY STUDY PLAN =====

Day 1: Mathematics - 1 hour (highest priority)

Day 1: Python - 1 hour

Day 2: English - 1 hour

===== PRIORITY ORDER =====

1. Mathematics

2. Python

3. English

13. RESULTS

The program produces a readable weekly timetable and a separate priority ranking based on the calculated utility value. The schedule demonstrates how the agent converts the input conditions into a decision. The selected ordering is directly traceable to the priority formula, making the agent's reasoning transparent and understandable.

14. LIMITATIONS

The current implementation assigns one hour per scheduled subject and handles at most seven timetable days. It does not estimate the total study time required for a subject, account for subject dependencies, or automatically reschedule missed sessions. These can be added in an extended version. The current rationality claim is specifically with respect to the defined priority objective and formula.

15. CONCLUSION

The AI-Based Study Planner demonstrates a simple rational-agent approach to study scheduling. By combining difficulty, topic weight and deadline urgency into a priority score, the system selects higher-priority topics first and converts the ordering into a weekly timetable. The explicit utility rule makes the decision process transparent and provides a clear justification for why the generated plan is rational given the supplied inputs.

16. REFERENCES

1. Russell, S. and Norvig, P., *Artificial Intelligence: A Modern Approach*.
2. Python Documentation, *datetime* — Basic date and time types.
3. Python Documentation, *Built-in Functions and Sorting*.
4. Project specification supplied for the AI-Based Study Planner rational-agent task.

APPENDIX – PYTHON IMPLEMENTATION

```
# AI-Based Study Planner - Rational Agent

from datetime import date

today = date.today()

n = int(input("Enter number of subjects: "))
subjects = []

for i in range(n):
    print(f"\nSubject {i + 1}")

    name = input("Subject name: ")
    deadline = input("Deadline (YYYY-MM-DD): ")
    difficulty = int(input("Difficulty (1-5): "))
    weight = int(input("Topic weight (1-5): "))

    deadline_date = date.fromisoformat(deadline)
    days_left = (deadline_date - today).days

    if days_left < 1:
        days_left = 1

    priority = (difficulty * weight) / days_left

    subjects.append({
        "name": name,
        "deadline": deadline,
        "difficulty": difficulty,
        "weight": weight,
        "priority": priority
    })

subjects.sort(key=lambda x: x["priority"], reverse=True)

hours = int(input("\nEnter available study hours per day: "))

print("\n===== WEEKLY STUDY PLAN =====")

day = 1
hour_used = 0

for subject in subjects:
    if hour_used >= hours:
        day += 1
        hour_used = 0

    if day > 7:
        break

    print(f"Day {day}: {subject['name']} - 1 hour "
          f"Priority: {subject['priority']:.2f}")

    hour_used += 1

print("\n===== PRIORITY ORDER =====")

for i, subject in enumerate(subjects, 1):
    print(
        f"{i}. {subject['name']} | "
        f"Priority = {subject['priority']:.2f}"
    )
```